

IoT-Based Intrusion Detection System Using Machine Learning

B. Tech. Project Report

Submitted by

Mahi Patel 612303107

Malhar Falke 612303108

Yugandhar Pise 612303140

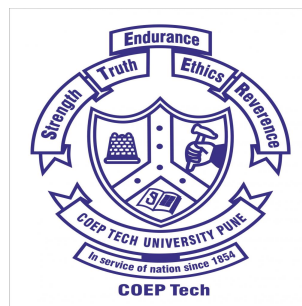
Vaibhav Tayade 612303184

Under the guidance of

Prof. S K Gaikwad

Department of Computer Engineering

COEP Technological University, Pune



DEPARTMENT OF COMPUTER ENGINEERING
COEP TECHNOLOGICAL UNIVERSITY, PUNE-5

September 2026

**DEPARTMENT OF COMPUTER ENGINEERING,
COEP TECHNOLOGICAL UNIVERSITY, PUNE-5**

CERTIFICATE

Certified that the B. Tech. project titled, “IoT-Based Intrusion Detection System Using Machine Learning” has been successfully completed by

Mahi Patel	612303107
Malhar Falke	612303108
Yugandhar Pise	612303140
Vaibhav Tayade	612303184

and is approved for the partial fulfillment of the requirements for the degree of “B. Tech. Computer Engineering”.

SIGNATURE

Prof. S K Gaikwad

Project Guide

Department of Computer Engineering

COEP Technological University,

Pune - 5.

SIGNATURE

Head of Department

Head

Department of Computer Engineering

COEP Technological University,

Pune - 5.

Abstract

This B. Tech. project report presents a live edge Intrusion Detection System (IDS) for Internet of Things (IoT) networks that runs on low-cost hardware. The system reads raw packets from a pcap file, timed replay, or live interface, groups packets into flows, and classifies each flow with a single 10-class XGBoost model exported to ONNX (about 92 KB). Training and live detection share one versioned 22-feature extractor (contract v2), so there is no train-serve mismatch. Flow verdicts are merged into per-source incidents, with an operator dashboard, syslog forwarding to SIEM in CEF or JSON, and an optional Intrusion Prevention (IPS) ladder of monitor, throttle, and block using nftables or iptables. The model is also exported to dependency-free C99 for microcontrollers.

On synthetic held-out traffic (about 12.2k flows from 70 scenarios unseen in training), the system reaches 99.65 percent accuracy with macro-F1 0.996, 100 percent attack detection rate, and 0 percent benign false positives. This result shows only that the pipeline works and the generated attacks are separable; it is not a claim of real-world accuracy. The hardest synthetic class is Mirai at 94.2 percent recall. All speed and memory figures in this report were measured on an Apple Silicon laptop: single-flow ONNX latency 10.5 microseconds (p99 16.2 microseconds), peak ONNX throughput about 407k flows per second, end-to-end pcap-to-verdict about 52.7k flows per second, and daemon memory 55.4 MB RSS. Raspberry Pi performance is not yet measured.

A second contribution is a correction. Two review passes over code and research found 26 findings, including a resubstitution error in the previous semester's in-domain evaluation where the model was tested on its own training rows. The affected headline numbers were withdrawn and archived under

legacy with a retraction notice. Earlier cross-dataset tests showed severe domain shift; a corrected rerun on 11 public datasets is pending. IPS per-class thresholds and abstention are implemented and tested, but their operating values are not yet validated on real deployment traffic.

Contents

List of Tables	7
List of Figures	8
List of Symbols	9
1 Introduction	1
1.1 Project Context	1
1.2 IoT Threats and Why Edge IDS	1
1.3 Objectives	2
1.4 Scope of This Report	2
1.5 Timeline	3
2 Literature Review	5
2.1 IDS Datasets	5
2.2 Machine Learning for IDS	6
2.3 Semantic Feature Alignment (SFAF)	7
2.4 Edge Deployment Literature	7
3 Research Gaps and Problem Statement	8
3.1 Research Gaps	8
3.1.1 Domain Shift Across Datasets	8
3.1.2 Train-Serve Mismatch	8

3.1.3	Edge Constraints	9
3.1.4	Evaluation Flaws	9
3.2	Problem Statement	9
3.3	Objectives for Evaluation	10
4	Proposed Methodology / Solution	11
4.1	System Overview	11
4.2	Packet Parser and Flow Table	11
4.3	Feature Contract v2 (22 Features)	12
4.4	Model: 10-Class XGBoost in ONNX	12
4.5	Incident Aggregation	13
4.6	Attack Generators and Scenario-Disjoint Data	13
4.7	Intrusion Prevention (IPS) Ladder	14
4.8	Dashboard, SIEM, and MCU Export	14
4.9	Pi Installer, Runbook, and Acceptance	15
5	Experimental Setup	16
5.1	Datasets	16
5.2	Tools and Reproduction	16
5.3	Benchmark Environment	17
5.4	Test and Quality Gates	17
5.5	Corrected Evaluation Protocol	18
6	Results and Discussion	19
6.1	Key Results With Evidence Scope	19
6.2	Discussion	19
6.3	Correction as a Result	21
6.4	Raspberry Pi Status	22

6.5	Result Figures	22
7	Conclusion	24
7.1	What Works	24
7.2	Correction as Contribution	25
7.3	Remaining Work for October Showcase	25
A	Demo Reproduction Guide	27
A.1	End-to-end demo	27
A.2	Full benchmark	27
A.3	Dashboard	28
A.4	Algorithms	28
B	Raspberry Pi Acceptance Checklist	30

List of Tables

1.1	Timeline, May–September 2026	4
4.1	Live model: 10 classes grouped by category	12
4.2	Feature contract v2 groups (22 features total)	13
5.1	Pi acceptance checklist status as on 30 September 2026	17
6.1	Key numbers with what each one actually proves	20
B.1	Pi acceptance details (all not yet measured)	30

List of Figures

4.1	Live edge IDS pipeline (logical blocks, no train-serve fork)	11
6.1	Confusion matrix on synthetic training-test split (10 classes)	22
6.2	Confusion matrix on synthetic held-out traffic from unseen scenarios	23
6.3	Inference latency and throughput measured on Apple Silicon laptop	23

List of Symbols

Symbol	Meaning
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
IoT	Internet of Things
ONNX	Open Neural Network Exchange (model format and run-time)
SFAF	Semantic Feature Alignment Framework (12-feature cross-dataset mapping)
CEF	Common Event Format (SIEM syslog format)
SIEM	Security Information and Event Management
SMOTE	Synthetic Minority Over-sampling Technique
RF	Random Forest
MCU	Microcontroller Unit
RSS	Resident Set Size (process memory)
p99	99th percentile latency
MQTT	Message Queuing Telemetry Transport (IoT messaging protocol)
nftables / iptables	Linux packet filtering frameworks used for IPS block/throttle
v2	Feature contract version 2 (22-feature live extractor)

Chapter 1

Introduction

1.1 Project Context

This report describes the project status of the project titled “IoT-Based Intrusion Detection System Using Machine Learning” as on 30 September 2026.

At the end of last semester (April 2026) the team had completed exploratory data analysis and baseline models (Random Forest, XGBoost with SMOTE) on public IDS datasets, a Semantic Feature Alignment Framework (SFAF) prototype that mapped different datasets into one shared 12-feature space, an ONNX edge model with a laptop latency benchmark, and the Semester 6 report [11, 9, 1, 4, 8].

The question for this period (May–September 2026) was to turn that prototype into a real, working IDS that runs on cheap edge hardware, and to check whether the earlier research claims hold up.

Repository: github.com/malharc373/IoT-IDS. Guide: Prof. S K Gaikwad, Department of Computer Engineering, COEP Technological University.

1.2 IoT Threats and Why Edge IDS

IoT devices are numerous, always connected, and rarely patched. Common attacks observed in public captures include port scans, SYN floods, UDP and

ICMP floods, SSH brute force, slowloris, Mirai botnet traffic, Xmas scans, and MQTT floods [6, 7, 5, 12, 10].

A cloud-only IDS adds latency, needs continuous uplink, and fails when the link fails. An edge IDS placed next to the IoT gateway can detect and optionally block attacks locally, with bounded memory and no dependency on connectivity. This motivates a small, fast model with a deterministic feature pipeline that fits on a Raspberry Pi and, in reduced form, on a microcontroller.

1.3 Objectives

- Build a live edge IDS that reads raw packets (pcap file, timed replay, live interface), extracts flow features, and classifies flows in real time.
- Use one shared, versioned feature extractor for both training and detection so that train-serve mismatch is impossible by construction.
- Provide operator tooling: incident aggregation, web dashboard, SIEM forwarding, and an optional IPS response ladder.
- Make the system deployable: one-command Raspberry Pi installer, runbook, acceptance checklist, and a microcontroller export.
- Correctly evaluate the system: scenario-disjoint held-out test on synthetic traffic, laptop benchmark, and a corrected cross-dataset protocol on 11 public datasets.

1.4 Scope of This Report

This report covers work done from 3 August to 5 September 2026, two review passes (19–22 August), and deployment tooling in progress on 30 Septem-

ber 2026. Raspberry Pi on-device measurements are not yet measured (see Chapters 5 and 6). The corrected 11-dataset cross-dataset rerun is pending. Remaining work for the October showcase is listed in Chapter 7.

1.5 Timeline

Table 1.1 summarises the period. There were no commits in the repository during May–July 2026. That period is recorded honestly as semester break with offline literature review, not as implementation work.

Table 1.1: Timeline, May–September 2026

Period	What happened
16 Apr 2026	Semester 6 report and presentation submitted
May–Jul 2026	Semester break / offline literature review (no commits in repo)
3 Aug 2026	Live edge IDS built: flow extractor, 10-class model, daemon, attack generators, Pi installer, IPS, MCU export, dashboard
4 Aug 2026	Expanded to 11 public datasets, cross-dataset study, full system benchmark
19–22 Aug 2026	Two full code and research reviews; 26 findings addressed; flawed earlier numbers withdrawn; corrected technical report dated 22 Aug
31 Aug 2026	Fixes merged to review branch; CI, CodeQL and Dependabot set up
5 Sep 2026	Laptop-side hardening: per-class thresholds, abstention, ESP32 sketch, type checking, fresh benchmark
30 Sep 2026	Raspberry Pi deployment in progress (installer and checklist ready; on-device numbers not yet measured)

Chapter 2

Literature Review

2.1 IDS Datasets

Public IDS datasets remain the standard way to train and compare detectors, but each covers a different network, era, and attack mix. This project uses 11 public datasets, each with its own loader.

- CICIDS2017 [11]: enterprise traffic with benign profiles plus brute force, DoS, web attacks, infiltration, and botnet. Widely used but known to be sensitive to train-test design.
- UNSW-NB15 [9]: synthetic attack testbed with nine attack families and Argus/Bro features. Useful for cross-dataset tests because its feature set differs from CICIDS2017.
- TON-IoT [1]: telemetry plus network and host logs from an IoT/IIoT testbed. Closer to IoT traffic than enterprise captures.
- IoT-23 [6]: large labelled IoT malware captures (Mirai, Torii, and others) totalling about 27 GB. The project loader was rewritten to handle this corpus in bounded memory, after finding that its space-separated label columns were being misread as entirely benign.

- Bot-IoT [7]: IoT botnet testbed with normal and attack traffic, including DDoS and information theft scenarios.
- WUSTL-IIoT [5]: industrial IoT testbed traffic for intrusion detection.
- CICDDoS2019 [12]: large-scale DDoS captures with many reflection and exploitation vectors.
- MQTT and related IoT captures [10]: publish-subscribe IoT messaging attacks including MQTT floods, relevant because many IoT deployments use MQTT.

Other loaders in the set cover additional IoT and enterprise captures. The key lesson from the literature is that a model trained on one dataset typically degrades on another (domain shift), so single-dataset accuracy is a weak claim.

2.2 Machine Learning for IDS

Random Forest and gradient boosting remain strong baselines for tabular flow features. Prior work reports competitive accuracy with careful preprocessing and class imbalance handling such as SMOTE [2, 4]. XGBoost is chosen here for the live model because it trains quickly, handles imbalanced classes, and exports cleanly to ONNX for edge inference [3, 8].

ONNX Runtime is the standard path for running the same model on laptop, Raspberry Pi, and constrained devices without rewriting inference code. Prior edge-IDS work shows that sub-100 KB boosted-tree models can meet microsecond-scale per-flow budgets on general-purpose CPUs, leaving headroom for parsing and flow-table upkeep.

2.3 Semantic Feature Alignment (SFAF)

SFAF maps different datasets (for example CICIDS2017, UNSW-NB15, TON-IoT) into one shared 12-feature space so that a single XGBoost model can train across them. The idea is to compare cross-dataset generalisation rather than single-dataset memorisation.

The status of SFAF in this report is restricted by the correction in Section 2.9 of the source progress file: last semester’s in-domain accuracy and “generalisation gain” headline numbers were withdrawn because of train-test overlap (resubstitution). The only safe qualitative statement at this stage is that earlier cross-dataset tests showed severe domain shift, and a corrected rerun is pending. No withdrawn numbers are cited in this report.

2.4 Edge Deployment Literature

Edge deployment studies emphasise three constraints: bounded memory, deterministic latency, and operability (install, update, monitor, recover). Common patterns are a systemd-supervised sensor plus dashboard, token-gated local web access, syslog forwarding to a central SIEM using CEF or JSON, and host or inline prevention with allowlists and dry-run defaults. The design in Chapter 4 follows these patterns. Microcontroller deployment typically requires a further reduction to dependency-free C with no heap allocation, as done here with the C99 export checked for parity against the ONNX model.

Chapter 3

Research Gaps and Problem Statement

3.1 Research Gaps

3.1.1 Domain Shift Across Datasets

Models that score well on one IDS dataset often fail on another network. Feature names, attack mixes, background traffic, and labelling all differ. Prior cross-dataset tests in this project showed severe domain shift. The literature confirms this is a general problem, not specific to one model. A detector that cannot be shown to transfer is of limited operational value.

3.1.2 Train-Serve Mismatch

Many published detectors train on precomputed CSV features but deploy on live packets with a separately written parser. Any difference in flow timeout, direction handling, TCP flag accounting, or normalisation silently invalidates the reported accuracy. The standard fix, adopted here, is a single feature-extraction codebase used in both paths with a versioned contract.

3.1.3 Edge Constraints

An edge IDS must parse packets at line rate, keep flow-table memory bounded, classify within microseconds, and run unattended (restart on boot, rotate logs, survive SIEM outages). Model size, stack use, and dependency footprint matter as much as accuracy. Laptop benchmarks do not transfer to the Pi; on-device measurement is required and is still pending in this project.

3.1.4 Evaluation Flaws

The most important gap found in this project was internal: last semester’s in-domain evaluation tested the model on the same rows it was trained on (resubstitution). This inflates accuracy and any “generalisation gain” built on it. Two further examples found during review were a cross-domain F1 metric that counted degenerate classifiers (for example a trivial “call everything an attack” classifier) as wins, and missing values silently dropped instead of tracked. All three were fixed by protocol changes described in Chapters 5 and 6.

3.2 Problem Statement

Design and demonstrate a live, edge-deployable intrusion detection system for IoT networks that (a) detects a defined set of network attacks in real time from raw packets, (b) runs within the memory and latency budget of a Raspberry Pi-class device, (c) provides operator tooling for incidents, dashboard, SIEM, and optional prevention, and (d) is evaluated honestly with disjoint train-test scenarios, explicit evidence scope for every number, and no train-test overlap.

3.3 Objectives for Evaluation

- Report synthetic-traffic accuracy only as “on synthetic held-out traffic” and never as real-world accuracy.
- Label every speed and memory figure as measured on Apple Silicon laptop until Pi numbers are measured.
- Never cite withdrawn SFAF headline numbers; report the correction itself as a result.
- Record Pi deployment rows as “not yet measured” where no on-device measurement exists, and do not estimate Pi performance by scaling laptop numbers.
- Treat IPS thresholds and abstention operating values as unvalidated on real deployment traffic until such traffic is tested.

Chapter 4

Proposed Methodology / Solution

4.1 System Overview

The live edge IDS follows a single pipeline: packets (pcap, replay or live) to parser to flow table to 22 features to ONNX model to per-source incidents to alert log, dashboard, SIEM and optional IPS block or throttle.

The research pipeline is separate: 11 public datasets to loaders to 12-feature alignment to cross-dataset evaluation.

Figure 4.1 is a logical block diagram of the live pipeline. Tables 4.1 and 4.2 list the model classes and feature groups.

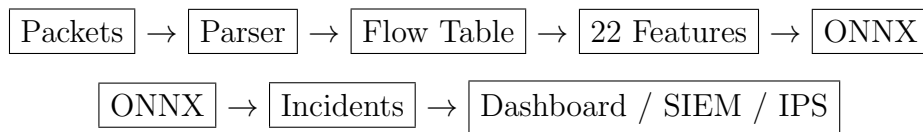


Figure 4.1: Live edge IDS pipeline (logical blocks, no train-serve fork)

4.2 Packet Parser and Flow Table

The sensor (flow feature extractor and detection daemon) supports three input modes: offline pcap file, timed replay, and live network interface.

The parser handles Ethernet, VLAN and QinQ, IPv4, IPv6, TCP, UDP, ICMP, and pcap plus pcapng capture formats. It reassembles IP fragments

Table 4.1: Live model: 10 classes grouped by category

Category	Classes
Benign	benign
Recon	portscan, Xmas scan
DoS	SYN flood, ICMP flood, UDP flood, slowloris, MQTT flood
Botnet	Mirai
Brute force	SSH brute force

safely and rejects capture formats it does not support instead of misreading them. Archive extraction fails closed against path traversal.

Packets are grouped into flows in a bounded flow table. Training and live detection call the exact same extractor, so there is no train-serve mismatch. Flow direction semantics were corrected during review, and a thread data race plus full-table rescoring on every tick were fixed.

4.3 Feature Contract v2 (22 Features)

One extractor produces a 22-feature flow vector, versioned as contract v2. The daemon refuses a model built for a different feature layout. Feature groups are summarised in Table 4.2.

Destination-port ablation (Chapter 6) shows accuracy unchanged with the port removed, confirming the model is not merely a port lookup.

4.4 Model: 10-Class XGBoost in ONNX

The live model is a 10-class XGBoost classifier exported to ONNX (about 92 KB, 91.8 KB measured). An earlier export used the wrong scaler; this was found in review and fixed, with export parity covered by tests.

Per-class confidence thresholds and an optional “unknown” abstention for

Table 4.2: Feature contract v2 groups (22 features total)

Group	Contents
Protocol and port	IP protocol, destination port
Duration and volume	flow duration, packet and byte counts, packet and byte rates
Packet length stats	mean, variance, min, max of packet lengths
Timing	inter-arrival mean and variance
TCP flags	ratios of SYN, ACK, FIN, RST, PSH, URG
Direction and context	forward/backward ratios, host-context counts

ambiguous predictions are implemented and tested. Their operating values have not been validated on real deployment traffic and must not be treated as tuned defaults.

4.5 Incident Aggregation

Per-flow verdicts are merged per attacker and attack type into incidents, so the operator sees one incident instead of thousands of per-flow lines. Aggregation state is bounded. Alerts go to a rotating alert log, the dashboard, and optionally the SIEM and IPS.

4.6 Attack Generators and Scenario-Disjoint Data

Traffic generators create each attack class plus realistic benign background traffic, including hard benign cases such as bursts and multi-host traffic that resemble attacks but are not.

The train-test split is by scenario: 280 train scenarios and 70 test scenarios, about 44.8k training flows and about 12.2k held-out flows. No scenario

appears in both sets. This is the basis for the “synthetic held-out” scope stated with every accuracy figure.

4.7 Intrusion Prevention (IPS) Ladder

The IPS module adds a response ladder of monitor, then throttle, then block, using nftables or iptables, for IPv4 and IPv6. It supports host mode (protect the Pi itself) and inline bridge mode (protect the whole network).

Safety defaults: dry-run mode, allowlists, per-class confidence thresholds, and optional abstention. Rate limiting that was documented but not implemented was found in review and has since been implemented. Blocking thresholds and abstention operating values are implemented and tested but not yet validated on real deployment traffic.

4.8 Dashboard, SIEM, and MCU Export

The dashboard shows live incidents and alerts and runs as a systemd service beside the sensor. It listens locally by default; network access requires a token stored in a mode-600 file. An earlier finding of unauthenticated network exposure has been fixed. Logs rotate.

SIEM forwarding sends incidents over syslog in CEF or JSON for Splunk, QRadar, Sentinel, or Elastic. A down SIEM does not crash the sensor.

The model is also exported to dependency-free C99 (102.7 KB header with about 43 KB constant data, about 130 bytes stack, zero heap). Output is checked against the original model. An ESP32 demo sketch is included.

4.9 Pi Installer, Runbook, and Acceptance

Deployment tooling is ready: one-command Pi installer (Python virtual environment, systemd services for sensor and dashboard, preflight check), Pi runbook covering passive versus inline placement, and Pi acceptance checklist covering benchmark, packet drops, memory, temperature, and 24-hour soak. On-device results are not yet measured; see Chapters 5 and 6.

Chapter 5

Experimental Setup

5.1 Datasets

Two kinds of data are used. They must not be confused.

- Synthetic scenario data for the live IDS: 280 train scenarios (about 44.8k flows) and 70 held-out test scenarios (about 12.2k flows), split by scenario so no scenario appears in both sets. Includes hard benign cases.
- Public captures for the research track: 11 public datasets including CICIDS2017, UNSW-NB15, TON-IoT, IoT-23, Bot-IoT, WUSTL-IIoT, CICDDoS2019, and MQTT-related captures, each with its own loader [11, 9, 1, 6, 7, 5, 12, 10].

IoT-23 handling is reported explicitly: the corpus is about 27 GB and its label columns are space-separated. An earlier loader misread it as entirely benign. The loader was rewritten for bounded-memory streaming.

5.2 Tools and Reproduction

- End-to-end demo script: generation of traffic, detection, and reporting.
- Benchmark script: full benchmark producing the benchmark report (5 September 2026 run is the source for Chapter 6).

- Operator dashboard for live incidents.
- Corrected technical report (22 August edition), data card, and model card.

5.3 Benchmark Environment

All speed and memory numbers in this report were measured on an Apple Silicon laptop. They are labelled as such in every table. Raspberry Pi numbers are not yet measured and are recorded as “not yet measured” in Table 5.1. No Pi estimate is made by scaling laptop numbers.

Table 5.1: Pi acceptance checklist status as on 30 September 2026

Item	Result
Pi model / OS	not yet measured
Installer ran cleanly	not yet measured
Sensor + dashboard services running	not yet measured
Live attack demo detected on Pi	not yet measured
Pi benchmark: latency / throughput	not yet measured
Memory / temperature / packet drops	not yet measured
24-hour soak	not done

5.4 Test and Quality Gates

67 automated tests cover parsing, flow state, thread safety, IPS rules, dashboard authentication, dataset safety, model export parity, and end-to-end demo. GitHub Actions CI runs ruff lint, tests, ONNX checks, C compilation, the demo, and the benchmark. Additional gates: mypy type checking on the live core, locked dependencies for development and Pi, Dependabot, CodeQL, and a security policy.

5.5 Corrected Evaluation Protocol

After the resubstitution finding, the protocol was fixed: proper 80/20 held-out splits per dataset, missing values tracked instead of silently dropped, memory-bounded random sampling, versioned caches, scenario-disjoint synthetic splits, and a threshold-transfer script that refuses to report wins from degenerate classifiers. Withdrawn results are kept in the legacy folder with a retraction notice and are not cited as valid.

Chapter 6

Results and Discussion

6.1 Key Results With Evidence Scope

Table 6.1 is the single authoritative result table. Every row carries its evidence scope. Source: benchmark report, 5 September 2026 run.

The synthetic 99.65 percent figure shows the pipeline works and the generated attacks are separable on synthetic held-out traffic. It is not real-world accuracy and is never presented as such. Attack detection at 100 percent with 0 percent benign false positives is likewise scoped to synthetic traffic that includes hard benign bursts.

6.2 Discussion

Mirai is the hardest synthetic class at 94.2 percent recall, which is consistent with botnet traffic that mimics benign IoT behaviour more closely than floods or scans. Port ablation leaves accuracy unchanged, increasing confidence that the classifier uses behavioural features rather than memorising ports.

Latency and throughput confirm headroom on the laptop: 10.5 microseconds mean and 16.2 microseconds p99 per flow, 407k flows per second in batch, 214k packets per second for parsing and aggregation, and 52.7k flows per second end to end. Daemon RSS is 55.4 MB measured on Apple Silicon

Table 6.1: Key numbers with what each one actually proves

Metric	Value	Evidence scope
Held-out accuracy, 10 classes	99.65%, macro-F1 0.996	Synthetic held-out only
Attack detection / benign FP	100% / 0%	Synthetic only
Hardest class (Mirai recall)	94.2%	Synthetic only
Accuracy without dest. port	Unchanged	Synthetic only; not port lookup
ONNX model size	91.8 KB	Artifact
C header for MCU	102.7 KB, 43 KB const, 130 B stack, 0 heap	Artifact
Single-flow ONNX latency	10.5 us (p99 16.2 us)	Apple Silicon laptop
Peak ONNX throughput	407k flows/s (batch 1024)	Laptop
Native C inference	1.16 us per flow	Laptop
Packet parse + aggregation	214k packets/s	Laptop
End-to-end pcap to verdicts	52.7k flows/s	Laptop
Daemon memory (RSS)	55.4 MB	Laptop
Tests	67	CI / local
Raspberry Pi performance	not yet measured	See Table 5.1
Cross-dataset metrics	withdrawn, rerun pending	Domain shift seen earlier

laptop. None of these predicts Pi performance. The Pi installer, services, and acceptance procedure are ready, but on-device latency, throughput, memory, temperature, drops, and soak remain not yet measured.

IPS blocking, per-class thresholds, and abstention are implemented and tested, including the previously missing rate limiting. Their operating values have not been validated on real deployment traffic, so no blocking precision or false-block rate is claimed.

6.3 Correction as a Result

Two review passes (19–22 August) produced 26 findings (F01–F26). Most were fixed; a few suspected problems were measured and shown not to be real; one was mitigated. Important examples: SFAF feature mappings were semantically wrong; the cross-domain F1 metric counted degenerate classifiers; the edge ONNX model used the wrong scaler; live mode had a thread data race and rescored the whole flow table each tick; IPS rate limiting was documented but not implemented; the dashboard was open to the network without authentication; flow direction semantics were wrong; dataset archive extraction allowed path traversal and now fails closed.

The most important research correction is the resubstitution error: last semester’s in-domain model was tested on the same rows it was trained on. Those headline numbers and the generalisation gain built on them were withdrawn, retained only in the legacy folder with a retraction notice, and are not cited in this report. The corrected protocol (80/20 held-out splits, missing-value tracking, versioned caches, scenario-disjoint synthetic splits) plus regression tests prevent recurrence. Earlier cross-dataset tests showed severe domain shift; a corrected rerun on all 11 datasets is pending.

6.4 Raspberry Pi Status

Deployment status is in progress. Software, installer, runbook, and checklist are complete. The on-device run was being attempted on 30 September 2026. All rows in Table 5.1 are not yet measured or not done. No estimate is offered.

6.5 Result Figures

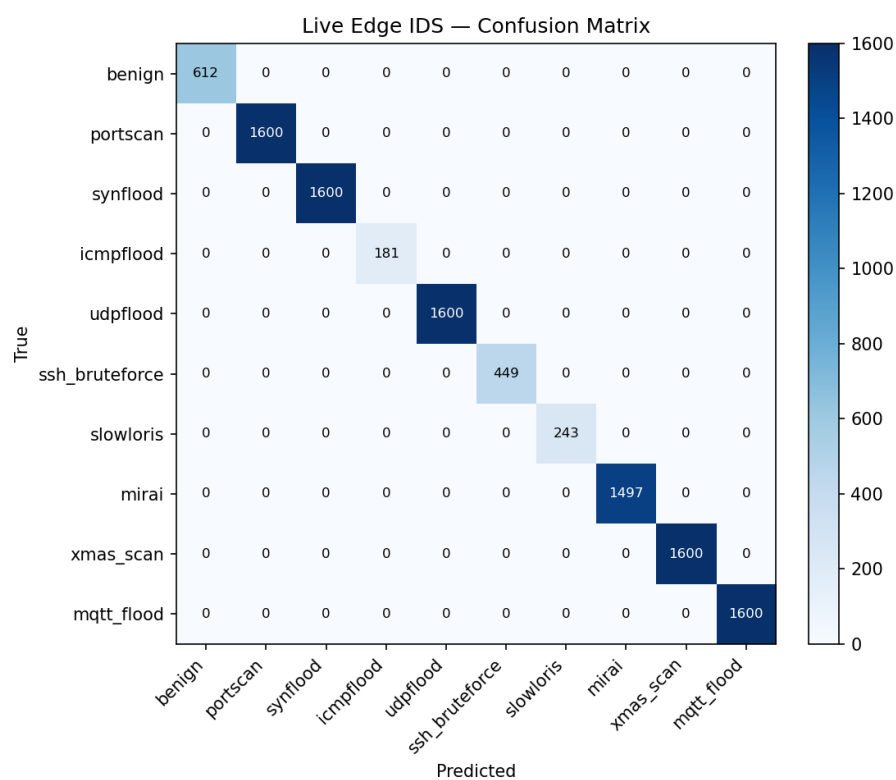


Figure 6.1: Confusion matrix on synthetic training-test split (10 classes)

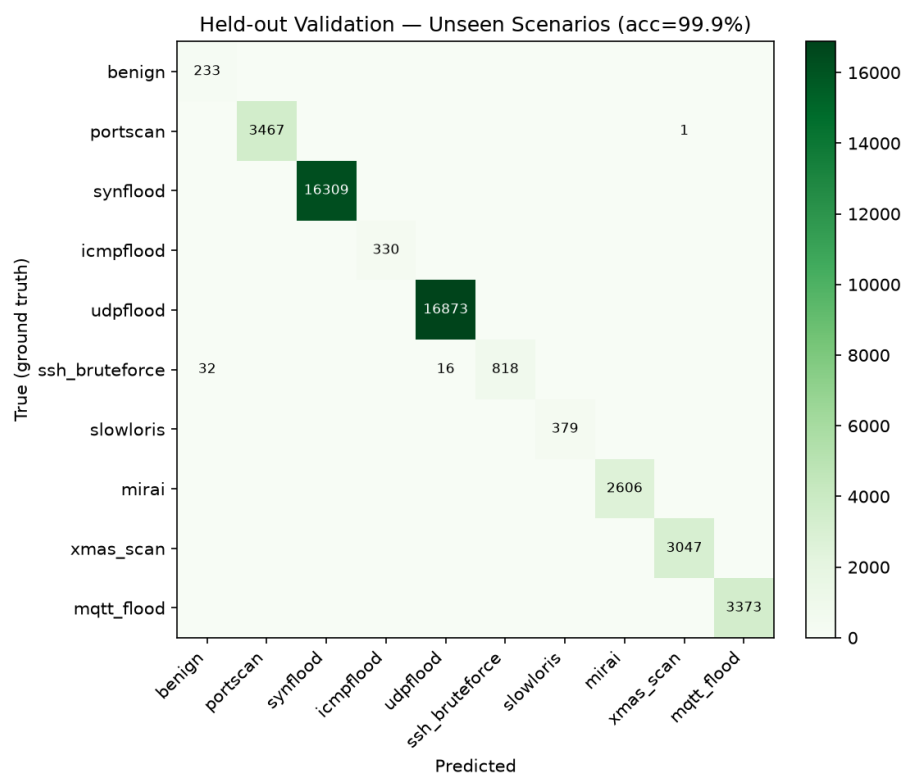


Figure 6.2: Confusion matrix on synthetic held-out traffic from unseen scenarios

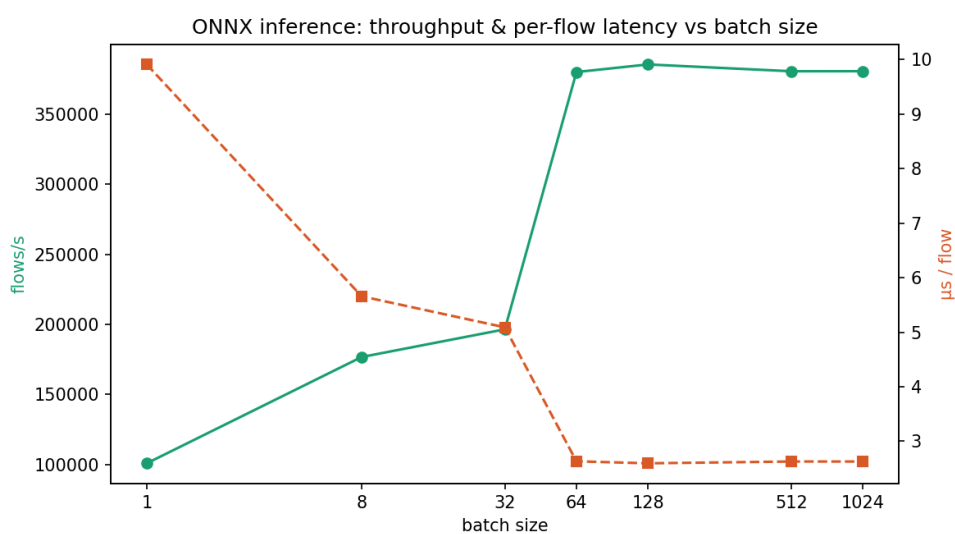


Figure 6.3: Inference latency and throughput measured on Apple Silicon laptop

Chapter 7

Conclusion

7.1 What Works

A live edge IDS now exists and runs end to end from raw packets to incidents: versioned 22-feature extraction shared by training and detection, a 10-class XGBoost ONNX model of about 92 KB, bounded incident aggregation, dashboard with token authentication and systemd supervision, CEF/JSON syslog forwarding that survives SIEM outage, an IPS ladder of monitor, throttle, and block for IPv4 and IPv6 in host and bridge modes, and a dependency-free C99 export with ESP32 sketch. Reproduction is one command with the end-to-end demo and benchmark scripts, guarded by 67 tests, CI, mypy, CodeQL, and locked dependencies.

Measured performance on synthetic held-out traffic is 99.65 percent accuracy (macro-F1 0.996) with Mirai the hardest class at 94.2 percent recall. This is scoped strictly to synthetic held-out traffic. Laptop performance is 10.5 microseconds mean per-flow latency and 52.7k flows per second end to end at 55.4 MB RSS, all measured on Apple Silicon laptop.

7.2 Correction as Contribution

The withdrawal of flawed earlier numbers is presented as a result. Finding a resubstitution error in our own evaluation, withdrawing the affected headlines, fixing the protocol, and adding tests so it cannot recur is a research contribution at B. Tech. level. The 26 review findings and their fixes are documented in the corrected technical report of 22 August.

7.3 Remaining Work for October Showcase

The final goal is to show the complete system running live: attacks launched from a laptop are detected on the Pi and shown on the dashboard, with optional blocking. Priority order:

- P0: Raspberry Pi deployment with benchmark and 24-hour soak, filling Table 5.1 with real numbers.
- P0: Real labelled IoT traffic (for example IoT-23 pcaps) through the live extractor, with per-class results on real traffic.
- P0: Corrected 11-dataset cross-dataset rerun with no train-test overlap, and merge of all branches to main.
- P1: Written decision on whether the research (12-feature) and live (22-feature) models should converge.
- P2: Drift, adversarial robustness, and calibration evaluation; licence choice and signed releases (post-showcase if time is short).

Week plan: 1–7 October Pi deployment and live demo reliable from fresh boot; 8–14 October Pi benchmark, soak, and real-traffic test; 15–21 October dataset rerun and main merge; 22–31 October final report, slides, backup

demo video, and rehearsal. If time runs short, cut from the bottom. The live Pi demo and honest numbers matter most.

Appendix A

Demo Reproduction Guide

This appendix lists the steps to reproduce the results in Chapter 6 from a fresh checkout.

A.1 End-to-end demo

Run the end-to-end demo script to generate synthetic traffic, run detection, and produce the detection report. This exercises the parser, flow table, feature extractor, ONNX model, and incident aggregation described in Chapter 4.

A.2 Full benchmark

Run the benchmark script to produce the benchmark report dated 5 September 2026. All latency, throughput, and memory figures in Table 6.1 come from this run on an Apple Silicon laptop. Do not scale these numbers to estimate Raspberry Pi performance.

A.3 Dashboard

Start the operator dashboard separately to view live incidents and alerts during the demo.

A.4 Algorithms

Algorithm A.1: Flow construction and 22-feature extraction.

1. Parse each raw packet into normalized fields (IP, ports, TCP flags, length, timestamp). Discard unparseable packets.
2. Compute flow key from 5-tuple. If key unseen, create new flow record.
3. Update flow record: packet and byte counts, length statistics, inter-arrival times, TCP flag ratios, direction ratios.
4. Update host context for source: distinct destination ports, distinct destination IPs, flow count in window.
5. On window expiry, emit 22-feature vector: proto, duration, tot_pkts, tot_bytes, pkts_per_sec, bytes_per_sec, mean/min/max/std pkt_len, mean/std iat, syn/fin/rst/ack ratios, fwd-bwd and down-up ratios, host_dst_ports, host_dst_ips, host_flow_count, dst_port.
6. Prune flows idle longer than timeout to bound memory.

Algorithm A.2: Real-time detection loop.

1. Load ONNX model and label map. Set confidence threshold and window length.
2. For each time step: collect expired flows, extract features with Algorithm A.1, run ONNX inference.
3. If confidence below threshold or abstention triggered, mark flow as unknown.
4. Aggregate verdicts per (source IP, attack category) into one incident. Up-

date alert log.

5. Forward incidents to dashboard and SIEM. If IPS enabled and confidence above per-class threshold and source not allowlisted, escalate monitor to throttle to block.

Algorithm A.3: IPS response ladder.

1. Default mode is dry-run: log intended action, change nothing.
2. On incident above threshold: check allowlist (loopback, sensor IP, operator subnets). If listed, take no action.
3. Else apply monitor, then throttle, then block with auto-expiry. Detect backend: nftables preferred, else iptables. On non-Linux or without root, remain in dry-run.
4. Expire blocks after timeout. Record all actions with timestamp and confidence.

Appendix B

Raspberry Pi Acceptance Checklist

This appendix expands Table 5.1. Status as on 30 September 2026. Deployment tooling (installer, runbook, checklist) is ready. On-device measurements are not yet measured.

Table B.1: Pi acceptance details (all not yet measured)

Item	Result as on 30 Sep 2026
Pi model and OS	not yet measured
Installer ran cleanly	not yet measured
Sensor and dashboard services running	not yet measured
Live attack demo detected on Pi	not yet measured
Pi benchmark latency and throughput	not yet measured
Memory, temperature, packet drops	not yet measured
24-hour soak	not done

Pass criteria for the October showcase: installer runs cleanly from fresh OS, sensor and dashboard run as services after reboot, live attacks from a laptop are detected and shown on the dashboard, benchmark and soak are recorded, and no packet drops or over-temperature events occur during soak.

Bibliography

- [1] A. Alhazmi et al. Ton-iot telemetry dataset for iot and iiot intrusion detection. *UNSW TON-IoT Datasets*, 2023. IoT/IIoT testbed with network, host and telemetry logs.
- [2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of KDD*, 2016.
- [4] Y. Chen et al. Xgboost with smote for imbalanced intrusion detection. *Applied Sciences / IDS literature*, 2023. Baseline approach: XGBoost plus SMOTE oversampling.
- [5] Mohamed Amine Ferrag, Othmane Friha, Djallel Hamouda, Leandros Maglaras, and Helge Janicke. Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning. *IEEE Access*, 10, 2022. Includes WUSTL-IIoT style industrial IoT testbed traffic.
- [6] Sebastian Garcia, Agustin Parmisano, and Maria Jose Vazquez. Iot-23: A labeled dataset with malicious and benign iot network traffic. Technical report, Stratosphere Laboratory, CTU Prague, 2020. Version 1.0.0, about 27 GB of Mirai, Torii and benign captures.

- [7] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. In *Future Generation Computer Systems*, volume 100, pages 779–796, 2019.
- [8] Microsoft. Onnx runtime: Cross-platform machine learning inferencing, 2024. Runtime used for 10-class XGBoost ONNX model on laptop and Pi.
- [9] Nour Moustafa and Jill Slay. Unsw-nb15: A comprehensive data set for network intrusion detection systems. *Military Communications and Information Systems Conference (MilCIS)*, 2015.
- [10] OASIS. Mqtt version 5.0 specification, 2019. Publish-subscribe IoT messaging protocol; flood attacks studied in this project.
- [11] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, Funchal, Portugal, 2018.
- [12] Iman Sharafaldin, Arash Habibi Lashkari, Saqib Hakak, and Ali A. Ghorbani. Developing realistic distributed denial of service (ddos) attack dataset and taxonomy. In *IEEE 53rd International Carnahan Conference on Security Technology*, 2019.